Optimize Anything — Technical Report

Assignment 8 · 14-Day Vibe-Coding Challenge · Optimization Techniques Lab

Akhil

November 2026

- Summary
- 1. Main case study (Track E): university exam timetabling
- 2. Track A — Continuous optimization
- 3. Track B — Discrete optimization
- 4. Track C — Quantum optimization (simulated)
- 5. Track D — Submodular, parallel and distributed optimization
- 6. Track F — Compiler optimization
- 7. Track G — xv6 operating-system optimization
- 8. Track H — DBMS optimization (PostgreSQL 16, TPC-H SF 0.1)
- 9. Track I — Quantum algorithms and QML (simulated)
- 10. Track J — Software-engineering optimization
- 11. Chapter 6 coverage
- 12. Answers to the final questions (Section 11 of the brief)
- 13. Verification at submission
- 14. Limitations and next steps
- Appendix A — Viva preparation

Summary

This report covers all ten compulsory tracks (A–J). The main case study (Track E) is **university exam timetabling**: place every exam in a time slot with no student clashes, while spreading each student's exams apart. It is tested on synthetic data and on seven real institutions from the Toronto benchmark.

Every track states an objective and a baseline, compares methods under the same data and stopping rules, and checks correctness independently of the code being tested. All code was written by prompting an AI tool (Claude) and is disclosed in `prompts/`, the AI tools register in `README.md`, and the commit prefixes (`[AI:claude]`).

Headline results:

Track	Baseline → optimized	Evidence
E Timetabling	first fit → DSatur-proximity + hill climbing: clash-free on 7/7 real Toronto institutions (one via an ILP repair) vs 1/7 for the baseline; 18–32% lower proximity cost on synthetic data	`tracks/E_exam_timetabling/results/`
A Continuous	GD → Newton / L-BFGS / Adam etc.: e.g. 2,695 vs 138 (Nesterov) vs 1 (Newton) iterations at κ = 1,000	`tracks/A_continuous/results/`
B Discrete	greedy → exact B&B / DP / ILP: greedy gap 0.005–3%, exactness costs exponential time on correlated items	`tracks/B_discrete/results/`
C Quantum opt.	random cut 0.68 → QAOA p = 4 0.93 approximation ratio (simulation); classical local search 0.95	`tracks/C_quantum_optimization/results/`
D Submodular / parallel	serial → 1.72× on 2 cores, 3.40× on 4 cores; greedy hits the exact optimum on coverage	`tracks/D_submodular_parallel_distributed/results/`
F Compiler	O0 → O2: 4.89× fewer cycles (geomean, 8 programs), 0 miscompiles in 300 random programs	`tracks/F_compiler/results/`
G xv6	round robin → MLFQ: −56% interactive turnaround; eager fork → COW: 14× faster fork	`tracks/G_xv6/results/`
H DBMS	primary keys only → budgeted index selection: TPC-H workload 51.6 s → 1.5–1.7 s (every method at ≥ 16 MB), identical results	`tracks/H_dbms/results/`
I Quantum alg. / QML	linear search 2,019 → Grover 50 queries at N = 4,096 (query count only); QML below classical	`tracks/I_quantum_algorithms_qml/results/`
J Software eng.	full suite 17 s → ILP-minimised + bottleneck fix 2.1 s of test time (3.1 s wall), same coverage	`tracks/J_software_engineering/results/`

A result is **measured** when it was timed on real hardware (Tracks D, E, F's compile times, G, H, J). It is **simulated** for all quantum results (Tracks C and I) and for Track D's scheduler

simulator. It is **theoretical** where a bound is quoted (Amdahl's law, the $1 - 1/e$ greedy guarantee, Grover's $\sqrt{N}$).

Reproducing. Each track has its own `README.md` (setup, `pytest -q`, `python run_experiments.py`) and a generated `results/SUMMARY.md`. Tests:

Track	Tests	Track	Tests
A	19	F	32 + mutation check
B	31	G	full xv6 usertests on 4 kernels × 2 CPU counts
C	13	H	9
D	22	I	33
E	18	J	15

All were run before submission (§13).

---

1. Main case study (Track E): university exam timetabling

1.1 The process and the evidence it needs optimizing

Each term a university must assign every exam to a time slot.

- **Hard constraint:** two exams sharing even one student cannot be in the same slot (a *clash*).
- **Soft goal:** students should not sit exams in back-to-back slots.

A common manual method walks the course list and puts each exam in the first free slot ("first fit"). On our synthetic instances this baseline **failed to find a clash-free timetable in 7 of 20 runs**, with 50–200 exams and slot counts that better orderings fill without clashes. When it did succeed, its spacing cost was 22–47% higher than the optimized methods. On real data from seven institutions (Toronto benchmark) it was clash-free for only one. That is the evidence of an optimization opportunity.

1.2 Formulation

- **Sets:** exams E, students S, slots $T = \{0, \ldots, |T|-1\}$.
- **Data:** w_{ij} = number of students taking both exams i and j.
- **Decision variables:** $x_{et} \in \{0, 1\}$, which is 1 if exam e is in slot t.
- **Hard constraints:**
  - $\Sigma_t x_{et} = 1$ for each exam (exactly one slot);
  - $x_{it} + x_{jt} \le 1$ for each conflicting pair (i, j) and each slot t (no clashes).
- **Objective (the Carter proximity cost, the standard measure for this benchmark):**

  $\min (1/|S|) \Sigma_{i<j} w_{ij} \cdot 2^{5-|t_i-t_j|}$, for $1 \le |t_i - t_j| \le 5$.

Exams one slot apart cost 16 per shared student, five apart cost 1, and six or more apart cost nothing.

- **Secondary problem:** the minimum number of slots, which is graph colouring.
- **Assumptions:**
  - Slots are equally long and evenly spaced.
  - There are no room-capacity constraints (the Toronto benchmark has none).
  - The proximity cost is a reasonable proxy for student stress.
- **Metric order:** feasibility first, then proximity cost, then runtime.

Every cost is recomputed by `timetabling/evaluate.py`, which is independent of the solvers. That independence caught a real bug (failure log F1): an ILP's reported objective was four times the true cost.

1.3 Strategy selection

Factor	This problem	Consequence
Variable type	binary (exam × slot)	discrete methods; gradient methods don't apply
Size	50–400 synthetic exams; real sets 81–381 exams, up to 4,360 students	exact ILP only for tiny cases; heuristics at scale
Complexity	NP-hard (contains graph colouring)	greedy + local search; ILP for bounds and small cases
Hardware	one CPU	single-threaded; restarts would parallelise trivially (Track D)

1.4 Methods

Method	Role
First fit by course id	**baseline**
Largest-degree first	greedy ordering
DSatur	greedy, colours the most-constrained exam first
DSatur-proximity	DSatur, plus choosing the cheapest legal slot
DSatur-proximity + hill climbing	local search: move one exam to a cheaper clash-free slot (first improvement) until no move helps; feasibility is never broken
Min-slots ILP	exact colouring with HiGHS
Proximity ILP	exact proximity model with HiGHS, cold and warm-started from the heuristic

1.5 Results

Synthetic instances (5 seeds per size; mean proximity cost per student, lower is better):

Exams	First fit (baseline)	DSatur	DSatur-proximity + hill climb
50	48.95 (feasible 3/5)	53.47	**33.56 ± 0.83**
100	31.96 (4/5)	33.90	**25.32 ± 0.77**
200	31.24 (1/5)	35.15	**25.65 ± 0.41**
400	28.87 (5/5)	32.91	**19.66 ± 0.25**

Real data: the Toronto benchmark (7 institutions, 5 seeds; cost ± std):

Instance (exams / students / slots)	First fit (baseline)	DSatur	DSatur-proximity + hill climb
ear-f-83 (190 / 1,125 / 24)	infeasible	67.38	**40.83 ± 0.24**
hec-s-92 (81 / 2,823 / 18)	infeasible	infeasible	infeasible; ILP + hill climb **15.79 ± 1.04**
lse-f-91 (381 / 2,726 / 18)	infeasible	infeasible	**13.00 ± 0.04**
sta-f-83 (139 / 611 / 13)	211.93	194.40	**168.95 ± 0.29**
tre-s-92 (261 / 4,360 / 23)	infeasible	15.92	**10.09** (best known 7.75)
ute-s-92 (184 / 2,749 / 10)	infeasible	58.82	**31.70 ± 0.31**
yor-f-83 (181 / 941 / 21)	infeasible	61.66	**46.30 ± 0.06** (best known 34.84)

- **The baseline fails on real data:** first fit gives a clash-free timetable for only **1 of 7 real institutions**.
- **The optimized method:** clash-free on 6 of 7 directly, in under 0.2 s each, and 20% cheaper than first fit where both work (sta-f-83).
- **The seventh (hec-s-92)** defeats every greedy construction. The min-slots ILP finds an 18-slot clash-free timetable in 20 s, and hill climbing then lowers its cost from 25.25 to 15.79. That is a hybrid of an exact method for feasibility and a heuristic for quality.
- **Best known results:** the Nottingham benchmark page lists 7.75 for tre-s-92 and 34.84 for yor-f-83. Ours are 30–33% higher, which is the expected price of simple single-exam moves against state-of-the-art metaheuristics run for minutes or hours.

Exact methods.

- **Min-slots ILP:** it proved the minimum colouring for 4 of 5 50-exam instances (10–11 slots, against the 14 available). At 100 exams it hit its time limit with feasible solutions.
- **Proximity ILP:** on 12–18-exam instances it improved the heuristic by 4–31% within 30 s, but **could not prove optimality**. The model's LP relaxation is weak: fractional x can make every proximity term zero, so the lower bound stays at 0.

Conclusion. Exact methods are right for tiny instances and for bounds. Greedy construction plus local search is right at real scale, and it is fast: under 0.4 s for 400 exams.

1.6 Trade-offs, limitations, next steps

- **Runtime vs quality:** DSatur-proximity plus hill climbing costs about 190× the baseline's runtime (0.33 s vs 0.002 s at 400 exams), which is negligible for a task run once a term.
- **Simple local search:** it uses single-exam moves only. Kempe-chain moves, simulated annealing or tabu search would close more of the gap to published results, at more code

and tuning.
- **Simplified model:** it has no rooms, no invigilators and no fixed-slot requests. Real deployment needs those constraints, which would make the ILP more attractive for small departments.

---

2. Track A — Continuous optimization

- **Objective:** compare first- and second-order methods on problems with a known optimum and on ML losses.
- **Baseline:** plain gradient descent with a tuned step size.
- **Correctness:** every gradient, Hessian, Hessian-vector product and the network's backpropagation is checked against autograd, an independent automatic-differentiation library (19 tests).
- **Code:** `tracks/A_continuous/contopt/`: 13 optimizers written from scratch in NumPy.

Experiment	Main result
Convergence vs theory	at κ = 1,000: GD 2,695 iterations, momentum 220, Nesterov 138, Newton 1. GD grows ∝ κ and momentum ∝ √κ, as theory predicts. Convex-not-strongly-convex fit k^−1.96 (theory k^−2).
Landscapes	pure Newton converges *to* a saddle point. On six-hump camel it ends at a saddle or maximum in 47% of starts; first-order methods never do.
ML as optimization (L2 logistic regression)	curvature methods (Newton, Fisher, L-BFGS, diagonal Hessian, AdaGrad) win by orders of magnitude in loss, yet test accuracy moves only 0.840 → 0.842
Gauss–Newton	from a far start it diverges; Levenberg–Marquardt converges in 12 iterations
Backpropagation	matches finite differences to 1.2×10^{-10} and is ~65× cheaper; it computes the gradient, the optimizer uses it
Step-size sensitivity	Adam stays within 10× of its best loss over 1.5–2 decades of step size; GD, momentum and Nesterov diverge just above their best
Noisy objectives	smaller batches give a higher noise floor; a 1/√k decay removes it (batch 1: 0.91 → 0.003)

Trade-off. Second-order methods need far fewer iterations, but each one costs a linear solve. Hessian-free Newton-CG and L-BFGS get most of the benefit at first-order memory cost.

---

3. Track B — Discrete optimization

- **Objective:** 0/1 knapsack (maximise value under a capacity), plus QUBO formulation and feature selection.
- **Baseline:** greedy by value/weight ratio.
- **Correctness:** all exact methods agree with each other and with brute force on small cases; 31 tests.

Method	Uncorrelated, n = 800	Strongly correlated, n = 100+
Greedy (baseline)	gap 0.009%, 0.4 ms	gap up to 2.9% (at n = 50)
DP	exact, pseudo-polynomial (time ∝ capacity)	16.7 s at C ≈ 3.2 M (n = 60)
Own branch-and-bound	exact, 0.02 s, 3,720 nodes	hits the 5 M-node limit from n = 100: usually finds the optimum (worst gap 0.075%) but can't prove it
ILP (HiGHS)	exact, 0.3 s	most robust, but exceeds 30 s at 400+ items

- **Instance class matters more than size.** On strongly correlated items every item has nearly the same ratio, so the fractional bound prunes almost nothing.
- **QUBO / Ising:** the penalty P must be at least about the largest item value. Below that the ground state is often infeasible; at P ≥ max value it was correct on 30/30 instances.
- **Feature selection:** greedy forward selection used 42 subset evaluations instead of 495, and lost only ~0.2 points of accuracy.

---

4. Track C — Quantum optimization (simulated)

- **Objective:** Max-Cut on Track B's 10-node graph with QAOA, and the H_2 ground-state energy with VQE.
- **Baselines:**
  - random cut;
  - classical local search (Track B);
  - brute force;
  - Hartree–Fock and exact diagonalisation for H_2.
- **Correctness:** 13 tests, including the parameter-shift gradient vs finite differences.
- **All results are statevector or shot simulations in PennyLane, not hardware.**

Method	Approximation ratio	P(optimal cut)
Random cut	0.679	0.008
QAOA p = 1 / 2 / 4	0.838 / 0.886 / 0.932	0.08 / 0.17 / 0.33
Local search (classical)	0.949	39% of runs exact

- **Optimizers:** at equal circuit budget, none of GD, Adam, SPSA or the quantum natural gradient wins everywhere. SPSA is cheapest per step: 2 circuits vs 117 for parameter-shift gradients.
- **Finite shots:** the spread of the estimate × √shots stays at 1.4–1.7, i.e. the noise follows 1/√shots.
- **Hardware noise:** at 5% depolarizing noise, p = 1 beats p = 3. Depth stops helping.
- **Barren plateaus:**
  - *Global cost:* gradient variance falls ~1,000× from 2 to 12 qubits.
  - *Local, shallow cost:* stays flat.
- **VQE (UCC ansatz):** matches the exact H_2 energy within 0.0002 mHa at 7 bond lengths.

Honest conclusion. These are correct demonstrations of the methods. Brute force over 2^{10} states is trivial classically, so none of them is a quantum advantage.

---

5. Track D — Submodular, parallel and distributed optimization

- **Code:** `tracks/D_submodular_parallel_distributed/distopt/`.
- **Tests:** 22.

Submodularity tested, not assumed. Random checks of diminishing returns (3,000 trials each):

Objective	Diminishing-returns violations	What greedy reaches
Weighted coverage	0	100% of the exact ILP optimum (k = 2–8)
Facility location	0	94–97% of optimum
Feature-selection R^2	970	**50%, below the 1 − 1/e = 63% guarantee**
Supermodular synergy	2,419	78–99% (78–84% for k ≥ 3)

- **When the guarantee holds:** only when its assumptions do.
- **Lazy greedy:** found the same sets with 71 instead of 292 evaluations. Its shortcut is valid only because coverage is submodular.

Parallel speed-up, measured on two machines:

Machine	p = 2	p = 4	Amdahl fit f (p ≤ cores)
Cloud container, 2 cores	1.72×	1.80× (oversubscribed)	0.84
MacBook Air VM, 4 cores	1.89×	**3.40×**	0.94

The 4-core run is a Linux VM on the student's MacBook Air, so it is measured, but not bare metal. Its CSV files are in `tracks/D_submodular_parallel_distributed/results/mac_4core/`.

- **Below Amdahl's prediction:** speed-ups fall short of what the parallel fraction allows, because process start-up and load imbalance are ignored by the formula.
- **With 30–40% serial code:** at most 1.41–1.76×, and the Amdahl limit is 1/(1−f) ≈ 2.6–3.3× for any number of cores.
- **Weak scaling** (problem grows with p) keeps improving, as Gustafson's law predicts.

Load balancing (48 tasks of growing size; makespan / ILP-optimal makespan):

Workers	Block (static)	Dynamic	LPT	Work stealing
4	1.86	1.08	1.00	1.03
16	2.15	1.41	1.04	1.18

Overheads (2-core machine):

- **Granularity:** 200,000 tiny tasks take 4.7 s one per message, against 0.03 s in chunks.

- **Synchronization:** a lock per increment is 21× slower than local counts.
- **Locality:** column-wise access is 6.8× slower per element.
- **Communication:** shipping data every step makes data-parallel training 29–40× slower than keeping it in the workers (8.9–9.2 s vs 0.22–0.32 s).
- **Locality bug (F16):** strided shards cost ~9× per product. On the 4-core VM even contiguous shards lose to serial, because those products are memory-bound.

Distributed learning (FedAvg).

- **Mixed (IID) clients:** 0.948 accuracy with 10 local epochs and 10× less communication (centralized: 0.967).
- **Clients that each hold about two digits:** long local training causes client drift, falling to 0.837.

Multi-task learning.

- **Equal weights:** fail outright on the small-scale task (error 1.04, no better than predicting the mean).
- **MGDA:** 0.24.
- **GradNorm:** 0.28, but its weights collapse to (2, 0).
- **Caveat:** MGDA on normalised gradients is exactly "average the unit gradients" (α = 0.5); on raw gradients it neglects the large task.

---

6. Track F — Compiler optimization

- **Code:** `tracks/F_compiler/tinyc/`, a small compiler for a C-like language, built from scratch.
- **Pipeline:** front end → IR → six optimization passes → register allocation → a VM that counts cycles.

Passes:

- constant propagation;
- copy propagation;
- local CSE;
- strength reduction;
- loop-invariant code motion (LICM);
- dead-code elimination (DCE).

Register allocation: greedy graph colouring vs an ILP (optimal spills).

Correctness:

- **Differential testing:** 300 random programs, every pass alone and in several orders, against an unoptimized interpreter.
- **Mutation check:** 5 seeded compiler bugs, each caught by a test. The first random-program fuzzer missed two real bugs it should have found (failure log F10).
- **32 tests.**

Level	Cycle speed-up (geomean, 8 programs)	Loads + stores
O0	1.00×	565,146
O1: greedy register allocation	3.44×	7,334
O2: + six passes	**4.89×**	2,108
O3: + ILP register allocation	4.91×	1,808

- **Passes work together:** each pass alone gives 0.97–1.03×; together they give 1.42×. Constant propagation is the most important.
- **LICM alone is a real trade-off:** it slows six of the eight programs by 4–16% through register pressure, and speeds one up by 21%.
- **ILP register allocation:** never worse than greedy, but within 0.7% of it at 40× the allocation time. That's why real compilers use heuristics.
- **Pass ordering:**
  - the worst order is 1.75× slower than the best on one program;
  - iterating to a fixpoint beats every single order;
  - a greedy "best pass now" ordering fails, because LICM's benefit only shows after constant propagation.
- **Concept learned:** an optimization is only valid if it preserves observable behaviour, including runtime errors. That rules out the textbook `x / 2^k → x >> k` rewrite for negative x under truncating division.

---

7. Track G — xv6 operating-system optimization

- **Code:** `tracks/G_xv6/patches/xv6-optimize.patch`, 1,257 lines against xv6-riscv `e93dabeb`.
- **Correctness:** all four kernels boot and pass the full `usertests` with 1 and 3 CPUs: original, MLFQ, COW and both. The patch also includes the `cowtest` program.

MLFQ scheduler:

- **Design:** three levels with quanta of 1, 2 and 4 ticks; demotion after a full quantum; preemption by higher-priority work; a boost every 50 ticks.

Workload	Round robin	MLFQ
Interactive turnaround, with 3 CPU hogs	10.02 s	**4.45 s (−56%)**
Short jobs arriving after long ones	1.82 s	**1.06 s (−42%)**
4 identical CPU-bound jobs (makespan)	8.02 s	8.07 s (no change; 62% fewer context switches)

Copy-on-write fork:

- **Design:** reference-counted pages; COW pages marked read-only; copies made in the page-fault handler and in `copyout`.

Workload	Eager copy	COW
Fork → child exits	25.0 ms	**1.73 ms (14×)**
Fork → exec	26.2 ms	3.76 ms (7×)
Fork → child writes every page	24.7 ms	**56.1 ms (2.3× slower)**
Live children of a 16 MB parent in 128 MB	6	≥ 40

Concepts learned:

- **Scheduling:** MLFQ approximates shortest-job-first without knowing job lengths.
- **Virtual memory:** COW trades one bulk copy for page faults (~55 µs each vs ~24 µs per bulk-copied page). It loses when the child writes everything.
- **Kernel writes:** they bypass the MMU, so `copyout` must break COW explicitly.

---

8. Track H — DBMS optimization (PostgreSQL 16, TPC-H SF 0.1)

- **Objective:** choose secondary indexes under a storage budget to minimise the workload's estimated cost.
- **Methods:**
  - knapsack DP and a ratio greedy, on single-index benefits;
  - a "what-if" greedy that asks the real planner via hypothetical indexes (hypopg);
  - an ILP.
- **Baseline:** primary keys only.
- **Correctness:** every configuration returned identical results for all 22 queries, checked by an order-preserving SHA-256 over every row; 9 tests.

	Default	ILP, 16 MB budget
Whole workload	51.6 s	**1.61 s (~32×)**
Q17 / Q20	21.7 s / 28.1 s	0.06 s / 0.035 s

At 16 MB and above every method lands at 1.48–1.71 s; the run-to-run spread is as large as the differences between methods.

Query plans: Q17 and Q20 run a correlated subquery once per outer row, each a sequential scan of `lineitem` (a Seq Scan inside a SubPlan in EXPLAIN). An index on `l_partkey` turns it into a bitmap index scan.

Method comparison (by the planner's estimate):

- **Knapsack DP double counts:** it builds both `(l_partkey, l_suppkey)` and `(l_partkey)`.
- **Greedy is short-sighted:** at 25 MB it can't afford the composite index after taking the cheaper one.
- **The ILP was best or tied at 16, 25 and 40 MB.** At 4 MB the ILP and the DP picked indexes that *raised* the estimated cost by 0.6%. Only the what-if greedy, which checks each step against the real planner, refused them.

- **Measured, the methods are indistinguishable** at 16 MB and above, except Q20 (12–14 ms with the composite index vs 28–42 ms without).

Estimates vs reality:

- **Index sizes:** hypopg overestimates B-tree sizes ~3×, because it ignores deduplication.
- **Query costs:** the planner thought Q17 was 90× cheaper than Q20; it took 75% as long.

Concept learned: a cost-based optimizer is only as good as its cost model.

---

9. Track I — Quantum algorithms and QML (simulated)

Algorithms:

- **Hand-written circuits:** Deutsch–Jozsa, Bernstein–Vazirani, Grover, QFT, QPE and Shor (order finding, N = 15). All 23 correctness checks pass.
- **Correctness:** 33 tests, including QFT vs PennyLane's built-in to 10^{-16}.

N	Grover queries	Linear search (mean)	Grover simulation time	Linear search time
256	12	127.6	46 ms	4.6 µs
4,096	50	2,019	38 s	89 µs

Noise: at 1% noise per gate, Grover's success falls 0.997 → 0.21, while shallow Bernstein–Vazirani keeps 0.85.

Annealing:

- **Simulated quantum annealing:** needs anneal times beyond $1/\text{gap}^2 \approx 520$ to reach 87% success.
- **Classical simulated annealing:** 100% in 100 sweeps (2 ms).

Embedding: a fully connected 40-variable problem needs 212 qubits on Pegasus and 521 on Chimera.

QML vs classical (5 seeds, same splits):

Dataset	Best classical	Variational classifier	Quantum kernel
Moons	RBF SVM 0.950 ± 0.011	0.877	0.893
Cancer (4 PCA features)	MLP 0.967 ± 0.038	0.890	0.853

- **Training time:** 15–83 s for the quantum models in simulation, against 1–300 ms for classical ones.
- **Circuit counts:** a VQC needs 35,000–70,000 circuit runs to train.

Comparison with parallel computing (Track D):

- **Parallel speed-up is measured and modest:** at most the number of cores, e.g. 3.40× on 4 cores. Any program can get it, but Amdahl's law caps it.

- **Quantum advantage is asymptotic:** fewer *queries* (1 vs $2^{n-1} + 1$ for DJ, $\sqrt{N}$ for Grover), but only for specific structures, and only on fault-tolerant hardware.
- **No wall-clock gain on a simulator:** simulating it costs 2^n, so every quantum wall-clock time here is worse than classical.

---

10. Track J — Software-engineering optimization

Subject: test-suite minimization and prioritization for more-itertools v11.2.1:

- an open-source library of ~7,300 lines;
- 765 tests (~21,000 subtests).

Formulation: weighted set cover.

- **Decision variables:** $x_t \in \{0, 1\}$ for each test.
- **Objective:** min $\Sigma c_t x_t$, where c_t is the measured time.
- **Constraints:** every one of the 1,960 statements and 2,937 branch arcs covered by the full suite stays covered.

Methods:

- random-order baseline;
- Chvátal greedy (count and per second, plus redundancy removal);
- exact ILP (HiGHS);
- LP lower bound.

Suite	Tests	Test time	Faults (mutants) still killed
Full	765	17.0 s	100% (960 of 1,116 mutants)
Random order until covered	367	15.7 s	—
Greedy (most new coverage)	302	14.7 s	92.8%
ILP, minimum count	296	14.9 s	92.3%
ILP, minimum time	300	14.4 s	89.3%
Random subsets of the same size	~300	—	69–72%

- **Profile first:** two tests take 73% of the time, and both are essential.
- **Why minimization saves only 15%:** it removes 61% of tests but only 15% of time.
- **Fixing the bottleneck:** shrinking the two tests' inputs, to the smallest sizes that keep their coverage and kill the same mutants, cut test time to 4.7 s. With the ILP selection: 2.1 s of test time and a steady 3.1 s wall time, against a 12–45 s full run.
- **The cost:**
  - *Lost fault detection:* coverage-preserving suites still miss 7–11% of detectable faults.
  - *Weaker stress test:* the threaded test now uses 10 threads instead of 100.
- **Prioritization:** ordering by new coverage per second finds 50% of faults after 0.06 s of test time (original order: 1.92 s).

---

11. Chapter 6 coverage

The full checklist, mapping every method to its code and experiment, is the table in the root `README.md` (“Chapter 6 coverage checklist”).

Area	Methods
Foundations	ML as optimization; convex vs non-convex; local/global minima; saddle points
Continuous	GD, SGD, mini-batch, momentum, Nesterov, AdaGrad, RMSProp, Adam, Newton, Gauss–Newton, backpropagation, Fisher / diagonal / low-rank curvature, Hessian-vector products, convergence rates, step-size tuning, noisy objectives
Discrete	greedy, ILP, branch-and-bound, dynamic programming, feature selection, QUBO / Ising
Quantum	QAOA, VQE, PQCs, parameter-shift, SPSA, quantum natural gradient, barren plateaus, shots, noise, Deutsch–Jozsa, Bernstein–Vazirani, Grover, QFT, QPE, Shor, annealing and embedding, VQC, quantum kernels
Submodular / parallel / distributed	diminishing returns, greedy guarantee, load balancing, makespan, work stealing, locality, synchronization, communication, data / model / pipeline parallelism, Amdahl, Gustafson, FedAvg, MGDA, GradNorm
Strategy	method selection by variable type, size and hardware (§1.3; Track J §10)

12. Answers to the final questions (Section 11 of the brief)

1. Process and evidence.

- **Process:** university exam timetabling.
- **Evidence:** the manual-style first-fit baseline failed to produce any clash-free timetable in 7 of 20 synthetic runs, and on 6 of 7 real Toronto institutions. When it succeeded, it left 22–47% more back-to-back pressure on students than DSatur-proximity with hill climbing.

2. Objective and constraints.

- **Variables:** binary x_et.
- **Hard constraints:** one slot per exam; no clash in any slot.
- **Objective:** the Carter proximity cost (§1.2), weighting each shared-student pair by 2^{5−gap}.
- **Secondary model:** minimum slots (graph colouring).
- **Other tracks:** each states its own formulation, e.g. set cover in J, budgeted index selection in H, makespan in D.

3. Methods and why. Discrete binary variables at a size where exact methods stall led to:

- **greedy construction:** first fit, largest degree, DSatur, DSatur-proximity;
- **local search:** hill climbing;
- **ILP with branch-and-bound:** for minimum slots, for tiny proximity instances, and for bounds.

Gradient methods were rejected because the variables aren't continuous. QUBO/annealing was rejected (Track I): embedding overhead is 5–13×, and classical simulated annealing already wins.

4. Optimized vs baseline.

- **Synthetic:** always feasible, against 13/20 for the baseline. Proximity cost 18–32% lower than first fit at every size, at under 0.4 s per run.
- **Real Toronto data:** clash-free on 6 of 7 institutions directly and on the seventh with an ILP repair, against 1 of 7 for the baseline. 20% cheaper where both are feasible. Still 30–33% above the best published results (§1.5).

5. Trade-offs, failures, limitations.

- **Exactness costs time:**
  - the proximity ILP can't prove optimality (its LP bound is 0);
  - the knapsack B&B stalls on correlated items;
  - ILP register allocation is 40× slower for ≤ 0.7% gain.
- **Optimizations with a losing case:**
  - COW fork is 2.3× slower when the child writes everything;
  - LICM alone slows six of eight programs;
  - test minimization loses 7–11% of fault detection.
- **Estimates mislead:** the planner and hypopg (H), and count-based APFD (J).
- **Quantum methods** never beat classical baselines on these sizes (C, I).
- **Failures:** 16 AI-generated errors are documented with fixes in `docs/failure_log.md`.

6. Correctness and reproducibility evidence.

- **Independent evaluators:**
  - E's `evaluate.py`;
  - autograd in A;
  - brute force for small cases in B, D, E and J;
  - differential testing against an unoptimized interpreter in F;
  - xv6 `usertests` in G;
  - SHA-256 of query results in H;
  - fresh coverage reruns and mutation testing in J.
- **Seeded randomness**, with repeated runs and spreads reported.
- **Pinned versions** in `requirements.txt` (PuLP 2.9.0, PennyLane 0.45.1).
- **Setup scripts** for external code (xv6 commit, TPC-H dbgen, more-itertools tag).
- **About 190 automated tests** in total, plus xv6 `usertests`, all passing at submission.

7. Adapting to changes.

- **Larger timetables (thousands of exams):** keep the greedy + local-search pipeline and add Kempe-chain moves and parallel restarts (independent seeds scale almost perfectly,

per Track D).

- **Rooms or invigilators:** add capacity constraints. Construction must then also choose rooms; the ILP becomes useful for small faculties.
- **More cores:** parallel restarts; the 4-core results in Track D show near-linear scaling when there is no serial part.
- **Different hardware:** with quantum hardware, QUBO/QAOA would still be limited by embedding and noise (Tracks C, I).

8. Compiler, xv6 and DBMS.

Track	Optimization implemented	Concept learned	Correctness shown by
Compiler	six IR passes and two register allocators	semantics preservation, including runtime errors; pass interactions and ordering	300 random programs against an unoptimized interpreter, plus seeded-bug mutation testing
xv6	MLFQ scheduling and COW fork	priority scheduling vs round robin; page-fault-driven virtual memory, reference counting, why `copyout` must break COW	full `usertests` and `cowtest` (incl. a page-leak check) on all four kernels with 1 and 3 CPUs
DBMS	budgeted index selection (ILP, DP, greedy, what-if greedy)	cost-based planning, correlated subqueries, cost-model error	identical result checksums for all 22 queries in every configuration

9. QML, quantum algorithms, parallel computing vs classical.

- **Quantum algorithms** give provable *query* advantages:
  - DJ and BV: 1 query;
  - Grover: √N, i.e. 50 vs 2,019 queries at N = 4,096.

  But simulation costs 2^n, and noise erases deep-circuit advantages (Grover 0.997 → 0.21 at 1% noise).
- **QML:** accuracy was at most on a par with logistic regression and below the best classical model, at roughly 90× to 70,000× the training time in simulation.
- **Parallel computing:** gives real, measured wall-clock gains, but they are bounded by core count and serial fraction, and eaten by communication and granularity overheads.

10. AI tools, how they differed, errors fixed.

- **Main tool:** Claude Opus 5.5 (claude.ai) for all tracks.
- **Tool comparison** (`docs/tool_comparison.md`): Claude Sonnet 5.5 and Claude Haiku 5.5 got the same CSE prompt in fresh contexts.
  - Both were correct first time (10/10 hazard tests, 0/300 miscompiles).
  - Both were ~1.9% more effective than the reference pass, because they numbered values through copies.
  - Sonnet answered in 23 s, Haiku in 54 s.
- **Errors found and fixed** (`docs/failure_log.md`):
  - misreported ILP objectives (F1);
  - ignored warm starts (F2);
  - an unnoticed library API break (F3);
  - a default MIP gap passed off as optimality (F4);

- a wrong circuit-execution counter (F5);
- a false comment (F6);
- code that contradicted its own correct docstring (F13);
- a non-converging annealer (F7);
- a benchmark calibration bias (F8);
- an ILP that could build useless indexes (F9);
- a fuzzer blind to bugs (F10);
- an ablation that didn't ablate (F11);
- a test set from a different task (F12);
- an empty statement-coverage set (F14);
- mutation runs that couldn't time out (F15);
- strided data shards that defeated data parallelism (F16).

---

13. Verification at submission

Track	Tests	Result
A	19	pass
B	31	pass
C	13	pass
D	22	pass
E	18	pass
F	32 + mutation check	pass
G	full `usertests` + `cowtest` on 4 kernels × 2 CPU counts	**8/8 ALL TESTS PASSED** (rerun at submission, 2.7–3.6 min each)
H	9	pass (PostgreSQL 16 + hypopg)
I	33	pass
J	15	pass

- **When:** every test suite in the table was run on 2026-11-01, after all of the final changes. This includes the xv6 `usertests` (see `tracks/G_xv6/results/correctness.csv`). The experiment results themselves come from the runs dated in each `SUMMARY.md`.
- **Track F:** the mutation check caught each of the 5 real mutants; the sixth is equivalent, as documented.
- **Track J:** its tests need its own conftest's `collect_ignore`. Without it, pytest would also collect the subject library's copies; this was fixed during the final check.
- **Track E on real data:** reproduced by `./get_data.sh && python run_experiments.py`.
- **Track D on 4 cores:** reproduced by rerunning Exp 3–6 on any 4-core machine.

14. Limitations and next steps

- **Small hardware:** most timings come from a 2-core cloud container. Track D was repeated on a 4-core VM on the student's laptop, but G and H would also benefit from bare-metal runs.

- **Simulation only:** quantum results are simulated; hardware runs would add noise and queueing time.
- **Simplified timetabling model:** no rooms or invigilators. The next step is room capacities and stronger local search (Kempe chains, simulated annealing) to approach published Toronto results.
- **Same-vendor tool comparison:** both tools are Claude models. A non-Claude tool (Codex or Gemini, which appear to be installed on the student's laptop) should be added; the commands are in `docs/tool_comparison.md`.

Appendix A — Viva preparation

These are the points from each track's prompt log that the student should be able to explain without an AI tool.

- **E: the proximity cost.** Two students sharing exams 1 slot apart cost $2 \times 16 = 32$. Divide by $|S|$ to get the per-student cost.
  - *DSatur:* picks the uncoloured exam with the most distinct colours among its neighbours (ties: the most uncoloured neighbours).
  - *Why the proximity ILP's bound is 0:* $x_{et} = 1/|T|$ spreads each exam across all slots, so every linearised proximity variable can be 0.
- **A: logistic-regression derivatives.** The gradient is $\nabla = X^T(\sigma(Xw) - y)/n + \lambda w$. The Hessian is $H = X^T D X/n + \lambda I$, with $D = \mathrm{diag}(\sigma(1-\sigma))$.
  - *Why pure Newton finds saddles:* it solves $\nabla f = 0$ and is indifferent to the sign of the curvature.
- **B: why B&B's bound is an upper bound.** The fractional (Dantzig) knapsack relaxation can only add value.
  - *DP recurrence:* $V[i][c] = \max(V[i-1][c], V[i-1][c-w_i] + v_i)$.
- **C: parameter shift.** $\partial\langle H\rangle/\partial\theta = [\langle H\rangle(\theta + \pi/2) - \langle H\rangle(\theta - \pi/2)]/2$ for gates generated by Pauli operators.
  - *SPSA:* needs 2 evaluations per step, whatever the dimension.
- **D: MGDA for two tasks.** $\alpha^* = (g_2 - g_1)\cdot g_2 \,/\, \|g_1 - g_2\|^2$, clipped to $[0, 1]$.
  - *For unit vectors:* $(1 - g_1\cdot g_2)/(2 - 2g_1\cdot g_2) = 1/2$.
  - *Amdahl's law:* $S(p) = 1/((1 - f) + f/p)$.
  - *Gustafson's law:* $S(p) = (1 - f) + fp$.
- **F: when `x/2^k → x >> k` is wrong.** For negative x, since division truncates toward zero but the shift floors: $-3/2 = -1$, while $-3 >> 1 = -2$.
  - *LICM correctness:* the hoisted instruction must be invariant, its destination defined once in the loop, and safe to execute even if the loop runs zero times.
- **G: what happens on a store to a COW page.**
  1. A page fault (scause 15).
  2. `cowfault` checks `PTE_COW`.
  3. If the reference count is 1, it re-enables writing.
  4. Otherwise it allocates and copies the page, remaps it writable, drops one reference and flushes the TLB.
- **H: why the correlated subquery is slow.** EXPLAIN shows the SubPlan re-running a Seq Scan on `lineitem` for every outer row; an index on `l_partkey` makes each run an index lookup.

- **I: Grover iteration count.** ⌊(π/4)√(N/M)⌋.
  - *Shor's success condition:* the order r is even and a^{r/2} ≢ −1 (mod N).
- **J: the set-cover ILP by hand.** Tests {a: r1, r2}, {b: r2, r3}, {c: r3}:
  - r1 forces a, and r3 needs b or c;
  - the optimum is {a, c} if c is cheaper than b.
  - *Essential tests:* the sole cover of some requirement, hence in every solution.